TP 3 – A Scene Of Information And Flexibility

JDBC · DAO · JPA & Spring Boot · Connection Pool

Nom Prénom : .....

Groupe : .....

ISEP – Bases de données / Java

Octobre 2026

Sommaire

1. Environnement de travail et base de données
2. Partie I – Communication avec la base (JDBC)
    1. Exercice 1 – Localisation des départements
    2. Exercice 2 – Move Department
    3. Exercice 3 – Affichage générique d'une table
    4. Exercice 4 – Sécurité
    5. Exercice 5 – PreparedStatement
    6. Exercice 6 – displayTable avec PreparedStatement
    7. Exercice 7 – J2EE (bonus)
3. Partie II – DAO
    1. Exercice 8 – Inconvénients de JDBC « brut »
    2. Exercice 9 – Bean Dept
    3. Exercice 10 – DeptDAO.find
    4. Exercice 11 – EmpDAO.find (récursif)
    5. Exercice 12 – DAOFactory
4. Partie III – Spring Boot & JPA
    1. III.1 – Initialisation du projet
    2. III.2 – Activation de JPA
    3. III.3 – Lecture des données
    4. III.4 – Création via JPA + Swagger
    5. III.5 – CRUD REST complet (code complet)
5. Partie IV – Pool de connexions (bonus)
6. Partie V – Générateur de DAO (bonus)
7. Conclusion

1. Environnement de travail et base de données

Élément	Version utilisée
SGBD	PostgreSQL 16 (local, base `isep`, utilisateur `postgres`)
JDK	OpenJDK 21
Driver JDBC	`postgresql-42.7.3.jar` (celui demandé dans l'énoncé)
Build	`javac` pour les parties I, II, IV, V ; Maven 3 pour la partie III
Spring Boot	2.7.18 (voir remarque en partie III), Hibernate 5.6, Springfox 2.9.2

La base utilisée est le schéma classique **EMP / DEPT** (avec les colonnes `efirst` et `tel` comme dans l'énoncé). Le script de création est fourni ci-dessous ; il est rejoué avant chaque série de tests pour repartir d'un état propre.

sql/schema.sql

```sql
-- Schéma EMP / DEPT utilisé pour le TP 3 (PostgreSQL)
DROP TABLE IF EXISTS emp;
DROP TABLE IF EXISTS dept;

CREATE TABLE dept (
    deptno  INTEGER PRIMARY KEY,
    dname   VARCHAR(14),
    loc     VARCHAR(13)
);

CREATE TABLE emp (
    empno    INTEGER PRIMARY KEY,
    ename    VARCHAR(10),
    efirst   VARCHAR(10),
    job      VARCHAR(9),
    mgr      INTEGER REFERENCES emp(empno),
    hiredate DATE,
    sal      INTEGER,
    comm     INTEGER,
    tel      VARCHAR(10),
    deptno   INTEGER REFERENCES dept(deptno)
);

INSERT INTO dept VALUES (10, 'ACCOUNTING', 'NEW YORK');
INSERT INTO dept VALUES (20, 'RESEARCH',   'DALLAS');
INSERT INTO dept VALUES (30, 'SALES',      'CHICAGO');
INSERT INTO dept VALUES (40, 'OPERATIONS', 'BOSTON');

INSERT INTO emp VALUES (7839, 'KING',   'GUY',    'PRESIDENT', NULL, '1981-11-17', 5000, NULL,
'0149545241', 10);
INSERT INTO emp VALUES (7698, 'BLAKE',  'BOB',    'MANAGER',   7839, '1981-05-01', 2850, NULL,
'0149545254', 30);
INSERT INTO emp VALUES (7782, 'CLARK',  'JOHN',   'MANAGER',   7839, '1981-06-09', 2450, NULL,
'0149545245', 10);
INSERT INTO emp VALUES (7566, 'JONES',  'JOHN',   'MANAGER',   7839, '1981-04-02', 2975, NULL,
'0149545456', 20);
INSERT INTO emp VALUES (7788, 'SCOTT',  'GUY',    'ANALYST',   7566, '1987-04-19', 3000, NULL,
'0149545249', 20);
INSERT INTO emp VALUES (7902, 'FORD',   'PAUL',   'ANALYST',   7566, '1981-12-03', 3000, NULL,
'0149545250', 20);
INSERT INTO emp VALUES (7369, 'SMITH',  'JOHN',   'CLERK',     7902, '1980-12-17',  800, NULL,
'0149545240', 20);
```

```
INSERT INTO emp VALUES (7499, 'ALLEN',  'BOB',    'SALESMAN',  7698, '1981-02-20', 1600,  300,
'0149547243', 30);
INSERT INTO emp VALUES (7521, 'WARD',   'PETER',  'SALESMAN',  7698, '1981-02-22', 1250,  500,
'0149545247', 30);
INSERT INTO emp VALUES (7654, 'MARTIN', 'JOE',    'SALESMAN',  7698, '1981-09-28', 1250, 1400,
'0149545784', 30);
INSERT INTO emp VALUES (7844, 'TURNER', 'PETER',  'SALESMAN',  7698, '1981-09-08', 1500,    0,
'0149548243', 30);
INSERT INTO emp VALUES (7876, 'ADAMS',  'JOSEPH', 'CLERK',     7788, '1987-05-23', 1100, NULL,
'0149555243', 20);
INSERT INTO emp VALUES (7900, 'JAMES',  'ANDY',   'CLERK',     7698, '1981-12-03',  950, NULL,
'0149545248', 30);
INSERT INTO emp VALUES (7934, 'MILLER', 'TOM',    'CLERK',     7782, '1982-01-23', 1300, NULL,
'0149545246', 10);
```

```
etudiant@isep: ~/TP3
etudiant@isep:~$ psql -h localhost -U postgres isep -c "\d emp" -c "SELECT * FROM dept"
                    Table "public.emp"
  Column  |         Type          | Collation | Nullable | Default
----------+-----------------------+-----------+----------+---------
 empno    | integer               |           | not null |
 ename    | character varying(10) |           |          |
 efirst   | character varying(10) |           |          |
 job      | character varying(9)  |           |          |
 mgr      | integer               |           |          |
 hiredate | date                  |           |          |
 sal      | integer               |           |          |
 comm     | integer               |           |          |
 tel      | character varying(10) |           |          |
 deptno   | integer               |           |          |
Indexes:
    "emp_pkey" PRIMARY KEY, btree (empno)
Foreign-key constraints:
    "emp_deptno_fkey" FOREIGN KEY (deptno) REFERENCES dept(deptno)
    "emp_mgr_fkey" FOREIGN KEY (mgr) REFERENCES emp(empno)
Referenced by:
    TABLE "emp" CONSTRAINT "emp_mgr_fkey" FOREIGN KEY (mgr) REFERENCES emp(empno)

 deptno |   dname    |   loc
--------+------------+----------
     10 | ACCOUNTING | NEW YORK
     20 | RESEARCH   | DALLAS
     30 | SALES      | CHICAGO
     40 | OPERATIONS | BOSTON
(4 rows)

etudiant@isep:~$
```

Figure 1 : structure de la table EMP et contenu de DEPT après exécution du script

Organisation du code rendu :

- `part1-jdbc/src/fr/isep/tp3/jdbc` : partie I (classe `Main`) ;
- `part1-jdbc/src/fr/isep/tp3/dao` : partie II (beans, DAO, factory) ;
- `part1-jdbc/src/fr/isep/tp3/pool` et `generator` : bonus parties IV et V ;
- `TP3_testjpa/` : projet Maven Spring Boot de la partie III.

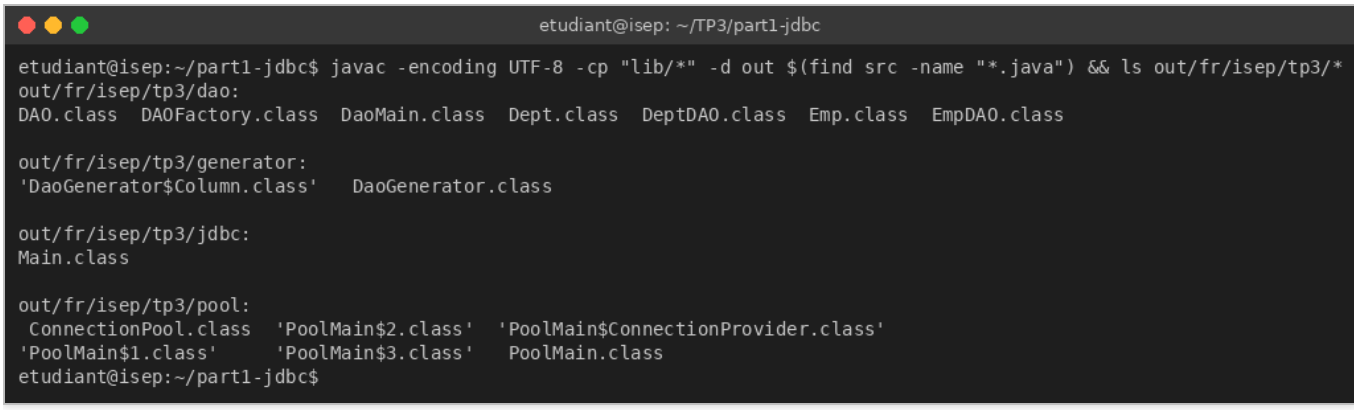


Figure 2 : compilation des parties I, II, IV et V en ligne de commande

2. Partie I – Communication avec la base (JDBC)

Le squelette de l'énoncé a été repris tel quel : chargement du driver avec `Class.forName("org.postgresql.Driver")` (facultatif depuis JDBC 4, le driver étant détecté automatiquement par le `ServiceLoader`, mais sans effet négatif), ouverture de la connexion via `DriverManager.getConnection(url, user, pass)` et fermeture dans le bloc `finally`. La méthode `main` prend en argument le numéro de l'exercice à exécuter, ce qui permet de lancer chaque exercice séparément.

part1-jdbc/src/fr/isep/tp3/jdbc/Main.java (extrait)

```
public static void main(String[] args) {
    /* Load JDBC Driver. */
    try {
        Class.forName("org.postgresql.Driver");
    } catch (ClassNotFoundException e) {
        e.printStackTrace();
    }

    String exercise = args.length > 0 ? args[0] : "ex1";
    Connection connexion = null;
    try {
        connexion = DriverManager.getConnection(URL, USER, PASS);
        System.out.println("Bdd Connected");

        switch (exercise) {
            case "ex1":
                displayDepartment(connexion);
                break;
            case "ex2": {
                Scanner sc = new Scanner(System.in);
                System.out.print("Employee number : ");
                int empno = sc.nextInt();
                System.out.print("New department number : ");
                int newDeptno = sc.nextInt();
                displayEmployee(connexion, empno);
                moveDepartment(connexion, empno, newDeptno);
                displayEmployee(connexion, empno);
                break;
            }
            case "ex3":
                displayTable(connexion, "emp");
                System.out.println();
                displayTable(connexion, "dept");
                break;
            case "ex4":
                sqlInjectionDemo(connexion);
                break;
            case "ex5": {
                Scanner sc = new Scanner(System.in);
                System.out.print("Employee number : ");
                String empno = sc.nextLine();
                System.out.print("New department number : ");
                String newDeptno = sc.nextLine();
                moveDepartmentPrepared(connexion, empno, newDeptno);
                displayTable(connexion, "emp");
                break;
            }
            case "ex6":
                displayTablePrepared(connexion, "emp");
                System.out.println();
                displayTableSafe(connexion, "dept");
```

```
                System.out.println();
                displayTableSafe(connexion, "emp; DROP TABLE emp");
                break;
            default:
                System.out.println("Unknown exercise " + exercise);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    } finally {
        if (connexion != null)
            try {
                connexion.close();
            } catch (SQLException ignore) {
                ignore.printStackTrace();
            }
    }
}
```

Exercice 1 – Ajout de la localisation du département

Il suffit d'ajouter la colonne `loc` dans le `SELECT` et de la récupérer avec `getString("loc")`. J'ai aussi ajouté la fermeture du `Statement`, oubliée dans le code d'exemple.

part1-jdbc/src/fr/isep/tp3/jdbc/Main.java (extrait)

```
/* Exercice 1 : affichage des départements avec leur localisation. */
public static void displayDepartment(Connection connexion) throws SQLException {
    Statement statement = connexion.createStatement();
    ResultSet resultat = statement.executeQuery("SELECT deptno, dname, loc FROM dept");

    while (resultat.next()) {
        int deptno = resultat.getInt("deptno");
        String dname = resultat.getString("dname");
        String loc = resultat.getString("loc");

        System.out.println("Department " + deptno + " is for "
                + dname + " and located in " + loc);
    }
    resultat.close();
    statement.close();
}
```

```
etudiant@isep: ~/TP3/part1-jdbc
etudiant@isep:~/part1-jdbc$ java -cp "out:lib/*" fr.isep.tp3.jdbc.Main ex1
Bdd Connected
Department 10 is for ACCOUNTING and located in NEW YORK
Department 20 is for RESEARCH and located in DALLAS
Department 30 is for SALES and located in CHICAGO
Department 40 is for OPERATIONS and located in BOSTON
etudiant@isep:~/part1-jdbc$
```

Figure 3 : exécution de l'exercice 1 : chaque département avec sa localisation

Exercice 2 – Move Department

La méthode exécute un `UPDATE` via `executeUpdate()`, qui renvoie le nombre de lignes modifiées (1 si l'employé existe, 0 sinon). Les valeurs sont saisies au clavier avec un `Scanner` (cf. Tips (1)). Une petite méthode `displayEmployee` affiche l'employé avant/après pour vérifier le résultat.

part1-jdbc/src/fr/isep/tp3/jdbc/Main.java (extrait)

```java
/* Exercice 2 : déplacement d'un employé (version Statement, vulnérable). */
public static void moveDepartment(Connection connexion, int empno, int newDeptno) throws
SQLException {
    Statement statement = connexion.createStatement();
    int nb = statement.executeUpdate(
            "UPDATE emp SET deptno = " + newDeptno + " WHERE empno = " + empno);
    System.out.println(nb + " employee(s) moved to department " + newDeptno);
    statement.close();
}
```

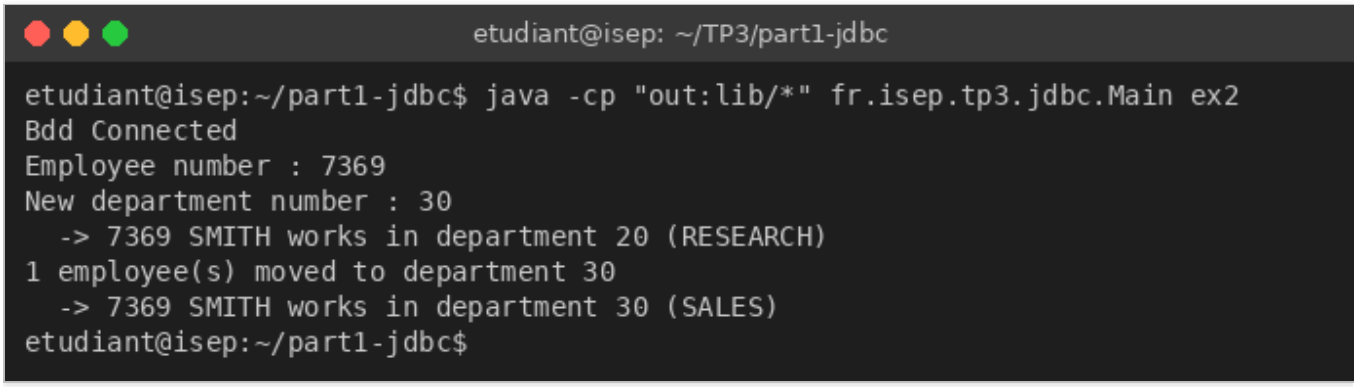


Figure 4 : SMITH (7369) passe du département 20 (RESEARCH) au département 30 (SALES)

Exercice 3 – Affichage générique d'une table

On ne connaît pas à l'avance les colonnes : on utilise donc `ResultSetMetaData` (Tips (2)). `getColumnCount()` donne le nombre de colonnes et `getColumnName(i)` leur nom, ce qui permet d'afficher l'en-tête, puis chaque ligne est parcourue colonne par colonne avec `getString(i)` (accepté même pour les colonnes numériques ou dates). L'affichage a été factorisé dans `printResultSet` car il est réutilisé dans les exercices suivants.

part1-jdbc/src/fr/isep/tp3/jdbc/Main.java (extrait)

```java
/* Exercice 3 : affichage générique d'une table grâce à ResultSetMetaData. */
public static void displayTable(Connection connexion, String tableName) throws SQLException {
    Statement statement = connexion.createStatement();
    ResultSet results = statement.executeQuery("SELECT * FROM " + tableName);
    printResultSet(results);
    results.close();
    statement.close();
}
```

part1-jdbc/src/fr/isep/tp3/jdbc/Main.java (extrait)

```java
static void printResultSet(ResultSet results) throws SQLException {
    ResultSetMetaData rsmd = results.getMetaData();
    int columnsNumber = rsmd.getColumnCount();

    StringBuilder header = new StringBuilder();
    for (int i = 1; i <= columnsNumber; i++) {
        header.append(rsmd.getColumnName(i).toUpperCase()).append(" | ");
    }
    System.out.println(header);

    while (results.next()) {
        StringBuilder line = new StringBuilder();
        for (int i = 1; i <= columnsNumber; i++) {
            line.append(results.getString(i)).append(" | ");
```

```
        }
        System.out.println(line);
    }
}
```

```
etudiant@isep: ~/TP3/part1-jdbc
etudiant@isep:~/part1-jdbc$ java -cp "out:lib/*" fr.isep.tp3.jdbc.Main ex3
Bdd Connected
EMPNO | ENAME | EFIRST | JOB | MGR | HIREDATE | SAL | COMM | TEL | DEPTNO |
7839 | KING | GUY | PRESIDENT | null | 1981-11-17 | 5000 | null | 0149545241 | 10 |
7698 | BLAKE | BOB | MANAGER | 7839 | 1981-05-01 | 2850 | null | 0149545254 | 30 |
7782 | CLARK | JOHN | MANAGER | 7839 | 1981-06-09 | 2450 | null | 0149545245 | 10 |
7566 | JONES | JOHN | MANAGER | 7839 | 1981-04-02 | 2975 | null | 0149545456 | 20 |
7788 | SCOTT | GUY | ANALYST | 7566 | 1987-04-19 | 3000 | null | 0149545249 | 20 |
7902 | FORD | PAUL | ANALYST | 7566 | 1981-12-03 | 3000 | null | 0149545250 | 20 |
7499 | ALLEN | BOB | SALESMAN | 7698 | 1981-02-20 | 1600 | 300 | 0149547243 | 30 |
7521 | WARD | PETER | SALESMAN | 7698 | 1981-02-22 | 1250 | 500 | 0149545247 | 30 |
7654 | MARTIN | JOE | SALESMAN | 7698 | 1981-09-28 | 1250 | 1400 | 0149545784 | 30 |
7844 | TURNER | PETER | SALESMAN | 7698 | 1981-09-08 | 1500 | 0 | 0149548243 | 30 |
7876 | ADAMS | JOSEPH | CLERK | 7788 | 1987-05-23 | 1100 | null | 0149555243 | 20 |
7900 | JAMES | ANDY | CLERK | 7698 | 1981-12-03 | 950 | null | 0149545248 | 30 |
7934 | MILLER | TOM | CLERK | 7782 | 1982-01-23 | 1300 | null | 0149545246 | 10 |
7369 | SMITH | JOHN | CLERK | 7902 | 1980-12-17 | 800 | null | 0149545240 | 30 |

DEPTNO | DNAME | LOC |
10 | ACCOUNTING | NEW YORK |
20 | RESEARCH | DALLAS |
30 | SALES | CHICAGO |
40 | OPERATIONS | BOSTON |
etudiant@isep:~/part1-jdbc$
```

Figure 5 : displayTable("emp") puis displayTable("dept") : même méthode, tables différentes

Exercice 4 – Sécurité

Faille principale : l'injection SQL. Les méthodes précédentes construisent la requête par **concaténation de chaînes** avec des valeurs venant de l'utilisateur. Le SGBD ne fait aucune différence entre le code SQL écrit par le développeur et les données saisies : un utilisateur peut donc saisir du SQL qui sera exécuté.

Démonstration (dans une transaction annulée par un `rollback` pour ne pas abîmer la base) :

- en saisissant `7369 OR 1=1` comme numéro d'employé, la requête devient `UPDATE emp SET deptno = 40 WHERE empno = 7369 OR 1=1` : **les 14 employés** sont déplacés au lieu d'un seul ;
- en saisissant `x' OR '1'='1` comme nom, la condition du `WHERE` est toujours vraie et on récupère toute la table (contournement d'une authentification, fuite de données comme les salaires) ;
- pour `displayTable(tableName)`, on pourrait même passer `emp; DROP TABLE emp`.

part1-jdbc/src/fr/isep/tp3/jdbc/Main.java (extrait)

```
/*
 * Exercice 4 : démonstration de l'injection SQL.
 * On simule une saisie utilisateur malveillante passée à une requête concaténée.
 * Tout est fait dans une transaction annulée pour ne pas abîmer la base.
 */
public static void sqlInjectionDemo(Connection connexion) throws SQLException {
```

```
    connexion.setAutoCommit(false);
    try {
        String empnoSaisi = "7369 OR 1=1";   // saisie de l'"utilisateur"
        String deptSaisi = "40";
        String sql = "UPDATE emp SET deptno = " + deptSaisi + " WHERE empno = " + empnoSaisi;
        System.out.println("Requête réellement envoyée : " + sql);
        Statement statement = connexion.createStatement();
        int nb = statement.executeUpdate(sql);
        System.out.println(nb + " employee(s) moved (au lieu de 1 !)");
        statement.close();

        String nomSaisi = "x' OR '1'='1";
        sql = "SELECT empno, ename, sal FROM emp WHERE ename = '" + nomSaisi + "'";
        System.out.println("\nRequête réellement envoyée : " + sql);
        statement = connexion.createStatement();
        ResultSet rs = statement.executeQuery(sql);
        printResultSet(rs);
        rs.close();
        statement.close();
    } finally {
        connexion.rollback();
        connexion.setAutoCommit(true);
        System.out.println("\n(rollback effectué, la base est intacte)");
    }
}
```

```
etudiant@isep: ~/TP3/part1-jdbc
etudiant@isep:~/part1-jdbc$ java -cp "out:lib/*" fr.isep.tp3.jdbc.Main ex4
Bdd Connected
Requête réellement envoyée : UPDATE emp SET deptno = 40 WHERE empno = 7369 OR 1=1
14 employee(s) moved (au lieu de 1 !)

Requête réellement envoyée : SELECT empno, ename, sal FROM emp WHERE ename = 'x' OR '1'='1'
EMPNO | ENAME | SAL |
7839 | KING | 5000 |
7698 | BLAKE | 2850 |
7782 | CLARK | 2450 |
7566 | JONES | 2975 |
7788 | SCOTT | 3000 |
7902 | FORD | 3000 |
7499 | ALLEN | 1600 |
7521 | WARD | 1250 |
7654 | MARTIN | 1250 |
7844 | TURNER | 1500 |
7876 | ADAMS | 1100 |
7900 | JAMES | 950 |
7934 | MILLER | 1300 |
7369 | SMITH | 800 |

(rollback effectué, la base est intacte)
etudiant@isep:~/part1-jdbc$
```

Figure 6 : l'injection modifie 14 lignes au lieu d'une et renvoie tous les salaires

Autres inconvénients de cette méthode « native » :

- **Performance** : avec un `Statement`, la requête est ré-analysée et re-planifiée par le SGBD à chaque exécution ;
- **Robustesse** : une valeur contenant une apostrophe (ex. `O'Neil`) casse la requête, il faut tout échapper à la main ;
- **Pas de typage** : les erreurs (nom de colonne, type) n'apparaissent qu'à l'exécution ;

- **Gestion manuelle des ressources** : `Connection`, `Statement`, `ResultSet` à fermer, sinon fuite de connexions et saturation du serveur ;
- **Coût de la connexion** : une connexion ouverte à chaque appel coûte cher (cf. partie IV) ;
- le SQL est mélangé au code métier et le mapping ligne → objet est entièrement manuel (cf. exercice 8).

Comme le rappelle l'énoncé, la validation doit se faire **côté serveur** : un contrôle en JavaScript côté client se contourne facilement.

Exercice 5 – moveDepartment avec PreparedStatement

La requête est écrite avec des `?` et les valeurs sont passées avec `setInt`. Le driver envoie la requête et les paramètres **séparément** au serveur : une valeur ne peut jamais être interprétée comme du SQL. Ici la saisie est lue comme une chaîne (pour pouvoir tenter l'injection) puis convertie en entier : une saisie non numérique est refusée avant même d'atteindre la base. J'utilise aussi un *try-with-resources* qui ferme automatiquement le `PreparedStatement`.

part1-jdbc/src/fr/isep/tp3/jdbc/Main.java (extrait)

```
/* Exercice 5 : moveDepartment avec PreparedStatement. */
public static void moveDepartmentPrepared(Connection connexion, String empno, String newDeptno) {
    String sql = "UPDATE emp SET deptno = ? WHERE empno = ?";
    try (PreparedStatement preparedStatement = connexion.prepareStatement(sql)) {
        preparedStatement.setInt(1, Integer.parseInt(newDeptno.trim()));
        preparedStatement.setInt(2, Integer.parseInt(empno.trim()));
        int nb = preparedStatement.executeUpdate();
        System.out.println(nb + " employee(s) moved to department " + newDeptno);
    } catch (NumberFormatException e) {
        System.out.println("Saisie refusée (pas un entier) : " + e.getMessage());
    } catch (SQLException e) {
        System.out.println("Erreur SQL : " + e.getMessage());
    }
}
```

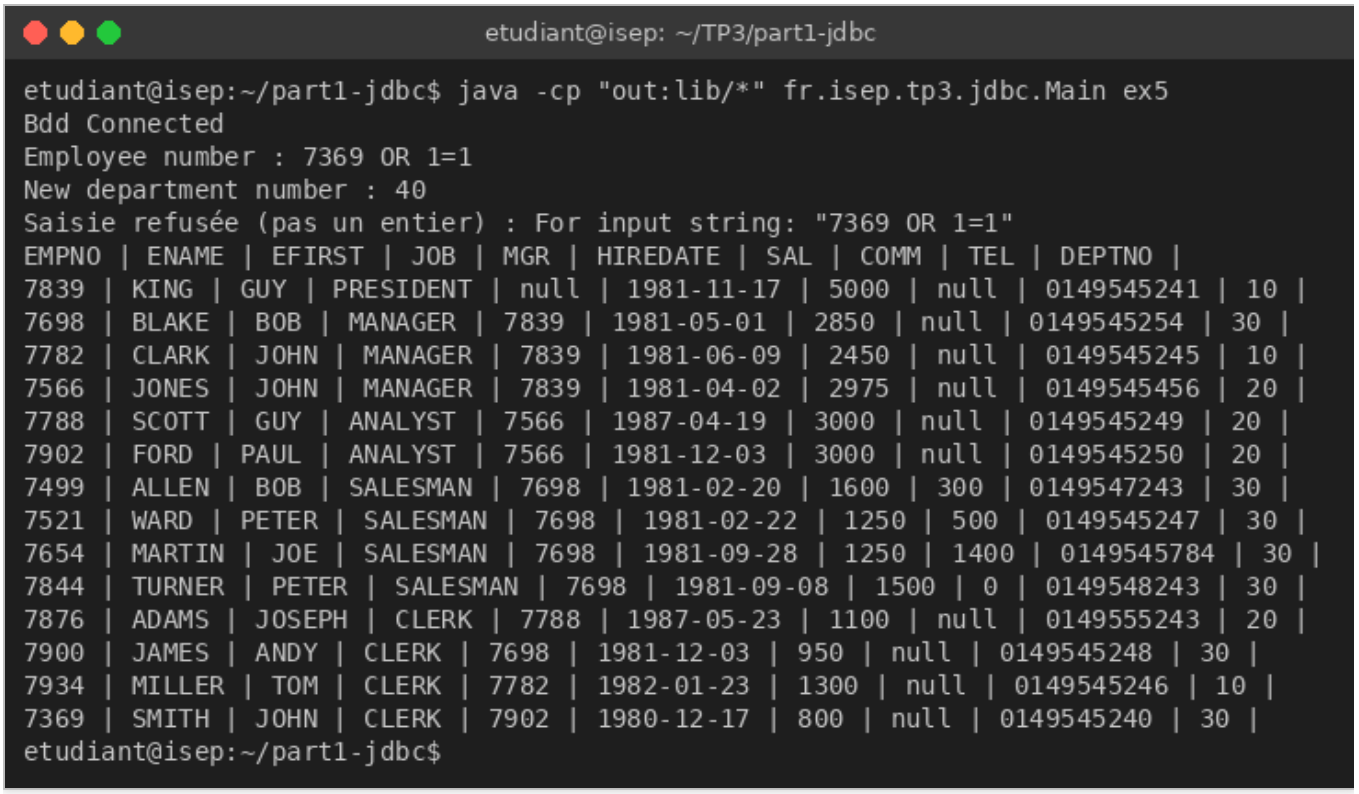


Figure 7 : la même attaque « 7369 OR 1=1 » est refusée, la table n'est pas modifiée

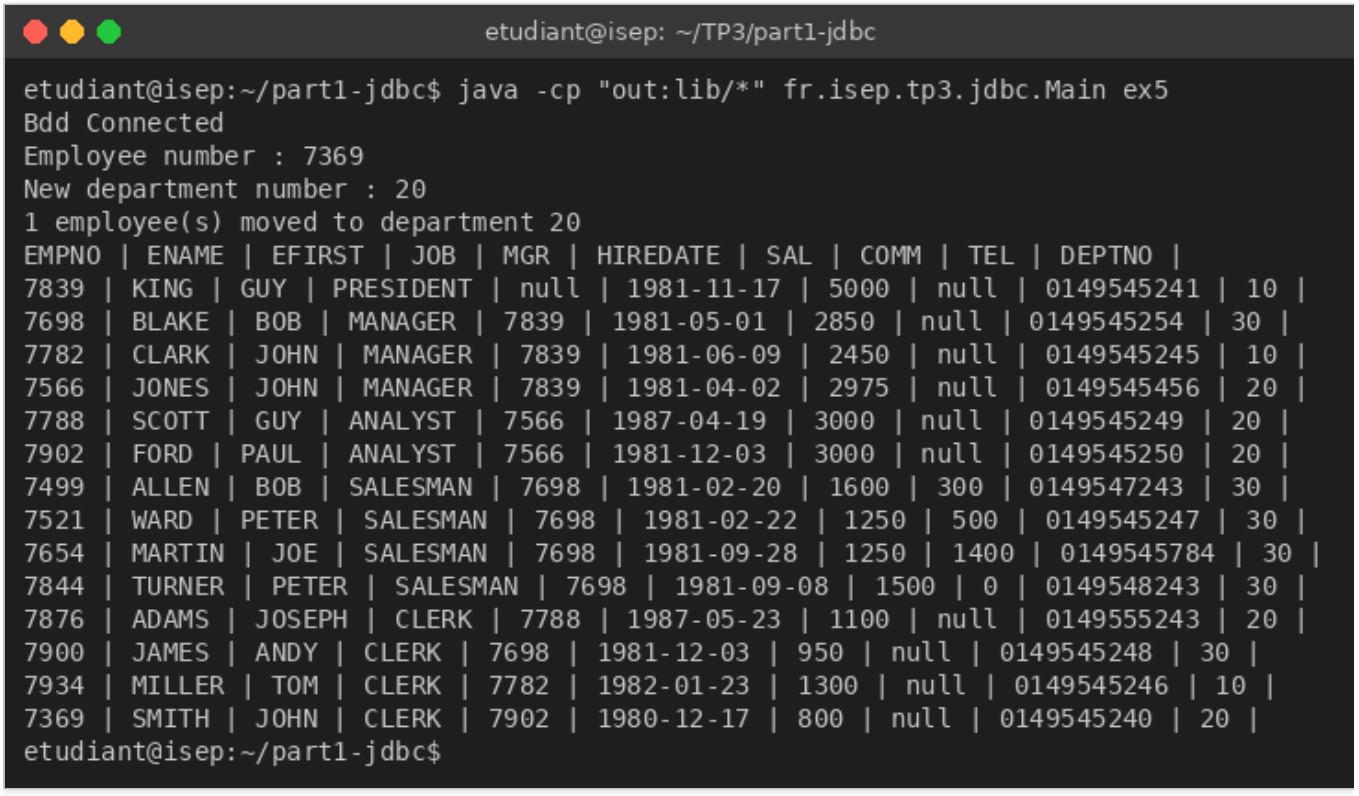


Figure 8 : utilisation normale : SMITH (7369) revient dans le département 20

Exercice 6 – displayTable avec PreparedStatement

part1-jdbc/src/fr/isep/tp3/jdbc/Main.java (extrait)

```java
/* Exercice 6 : displayTable avec PreparedStatement -> ne fonctionne pas. */
public static void displayTablePrepared(Connection connexion, String tableName) {
```

```
    try (PreparedStatement ps = connexion.prepareStatement("SELECT * FROM ?")) {
        ps.setString(1, tableName);
        ResultSet rs = ps.executeQuery();
        printResultSet(rs);
    } catch (SQLException e) {
        System.out.println("SQLException : " + e.getMessage());
        System.out.println("SQLState : " + e.getSQLState());
    }
}
```

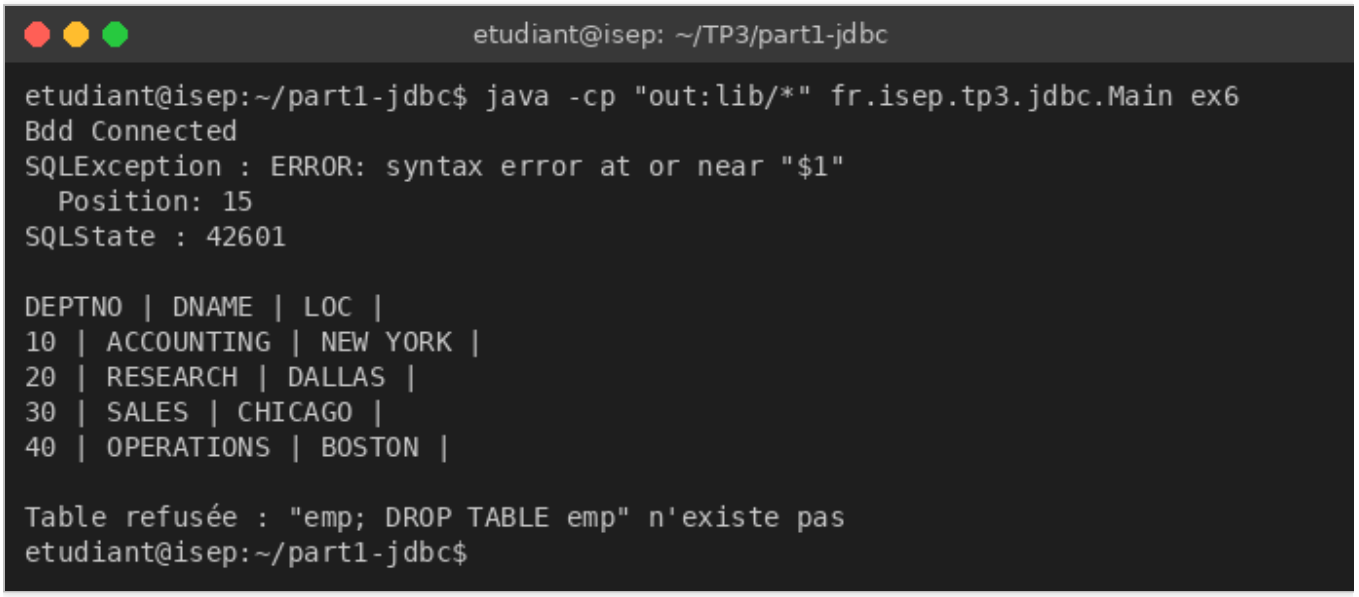


*Figure 9 : « SELECT * FROM ? » provoque une erreur de syntaxe ; la version sécurisée fonctionne et refuse l'injection*

> **Pourquoi ça ne marche pas ?** Les paramètres `?` d'une requête préparée ne peuvent remplacer que des **valeurs** (littéraux), jamais des **identifiants** (nom de table, de colonne) ni des mots-clés SQL. La requête est envoyée au serveur pour être analysée et planifiée **avant** que les valeurs soient connues : pour construire le plan d'exécution, PostgreSQL doit savoir quelle table lire. `SELECT * FROM $1` n'est donc pas du SQL valide (d'où l'erreur `syntax error at or near "$1"`, SQLState 42601). Et même si c'était accepté, `setString` produirait la chaîne littérale `'emp'` et non l'identifiant `emp`.
>
> **Solution** : vérifier le nom de table par **liste blanche**. J'interroge les métadonnées de la base (`DatabaseMetaData.getTables`) et je n'exécute la requête que si la table existe réellement.

part1-jdbc/src/fr/isep/tp3/jdbc/Main.java (extrait)

```
/*
 * Solution pour l'exercice 6 : un nom de table ne peut pas être un paramètre,
 * on le vérifie donc contre la liste des tables réelles (DatabaseMetaData) avant de l'utiliser.
 */
public static void displayTableSafe(Connection connexion, String tableName) throws SQLException {
    boolean exists;
    try (ResultSet tables = connexion.getMetaData().getTables(null, "public", tableName, new
String[]{"TABLE"})) {
        exists = tables.next();
    }
    if (!exists) {
        System.out.println("Table refusée : \"" + tableName + "\" n'existe pas");
        return;
    }
    displayTable(connexion, tableName);
}
```

Exercice 7 – J2EE (bonus)

Je n'ai pas réalisé cette version servlet/J2EE. L'exposition des données dans un navigateur est en revanche faite dans la partie III avec Spring Boot (Tomcat embarqué), qui est l'approche moderne de la même idée : le serveur reçoit les requêtes HTTP et interroge la base.

3. Partie II – DAO

Exercice 8 – Inconvénients de JDBC tel qu'utilisé ci-dessus

- **Code de persistance dispersé dans le code métier** : le SQL est écrit partout où on en a besoin (schéma « Before DAO »). Si une table change (colonne renommée, changement de SGBD), il faut modifier le code à de nombreux endroits ;
- **Pas d'objets métier** : on manipule des `ResultSet` et des colonnes au lieu d'objets `Emp` / `Dept` ; le mapping relationnel → objet est refait à chaque requête ;
- **Beaucoup de code répétitif** (« boilerplate ») : ouverture, `try/catch/finally`, fermeture des ressources, `SQLException` partout ;
- **Couplage fort** au SGBD et au driver : impossible de changer de source de données (autre base, fichier, web service) sans réécrire le métier ;
- **Difficile à tester** : on ne peut pas remplacer la base par un faux objet (mock) pour tester la logique métier ;
- gestion manuelle des connexions (une par appel), des transactions et des relations entre tables (ex. le manager d'un employé).

Le pattern **DAO** répond à cela en isolant toute la persistance dans une couche dédiée : le métier ne manipule que des objets et des méthodes `find / create / update / delete`, sans savoir comment ni où les données sont stockées.

Exercice 9 – Bean Dept

Un attribut par colonne de la table `DEPT`, avec le même type, un constructeur vide (convention JavaBean), les getters/setters et un `toString()` pour l'affichage.

part1-jdbc/src/fr/isep/tp3/dao/Dept.java

```java
package fr.isep.tp3.dao;

/* Exercice 9 : Bean (POJO) correspondant à la table DEPT. */
public class Dept {

    private int deptNo;

    private String dname;

    private String loc;

    public Dept() {
    }

    public Dept(int deptNo, String dname, String loc) {
        this.deptNo = deptNo;
        this.dname = dname;
        this.loc = loc;
    }

    public int getDeptNo() {
        return deptNo;
    }
```

```
    public void setDeptNo(int deptNo) {
        this.deptNo = deptNo;
    }

    public String getDname() {
        return dname;
    }

    public void setDname(String dname) {
        this.dname = dname;
    }

    public String getLoc() {
        return loc;
    }

    public void setLoc(String loc) {
        this.loc = loc;
    }

    @Override
    public String toString() {
        return "Dept{deptNo=" + deptNo + ", dname='" + dname + "', loc='" + loc + "'}";
    }
}
```

Le bean `Emp` fourni a été complété (getters/setters et `toString`). Le manager est bien de type `Emp` et le département de type `Dept`. J'ai choisi `String` pour `tel` au lieu de `int` : un numéro qui commence par 0 (`0149545241`) perdrait son zéro et ne tiendrait de toute façon pas dans un `int` pour certains numéros.

part1-jdbc/src/fr/isep/tp3/dao/Emp.java (extrait)

```
/* Bean (POJO) correspondant à la table EMP (fourni dans l'énoncé, complété). */
public class Emp {

    private Long empNo;

    private String ename;

    private String efirst;

    private String job;

    private Emp mgr;

    private Date hireDate;

    private int sal;

    private int comm;

    private String tel;

    private Dept department;

    public Long getEmpNo() {
        return empNo;
    }

    public void setEmpNo(Long empNo) {
        this.empNo = empNo;
    }

    public String getEname() {
```

```
        return ename;
    }

    public void setEname(String ename) {
        this.ename = ename;
    }

    public String getEfirst() {
        return efirst;
    }

    public void setEfirst(String efirst) {
        this.efirst = efirst;
    }

    public String getJob() {
        return job;
    }

    public void setJob(String job) {
        this.job = job;
    }

    public Emp getMgr() {
        return mgr;
    }

    public void setMgr(Emp mgr) {
        this.mgr = mgr;
    }

    public Date getHireDate() {
        return hireDate;
    }

    public void setHireDate(Date hireDate) {
        this.hireDate = hireDate;
    }

    public int getSal() {
        return sal;
    }

    public void setSal(int sal) {
        this.sal = sal;
    }

    public int getComm() {
        return comm;
    }

    public void setComm(int comm) {
        this.comm = comm;
    }

    public String getTel() {
        return tel;
    }

    public void setTel(String tel) {
        this.tel = tel;
    }

    public Dept getDepartment() {
        return department;
    }
```

```
    public void setDepartment(Dept department) {
        this.department = department;
    }

    @Override
    public String toString() {
        // on n'affiche que le nom du manager pour éviter un affichage récursif énorme
        return "Emp{empNo=" + empNo + ", ename='" + ename + "', efirst='" + efirst + "', job='" +
job
                + "', mgr=" + (mgr == null ? "null" : mgr.getEmpNo() + " " + mgr.getEname())
                + ", hireDate=" + hireDate + ", sal=" + sal + ", comm=" + comm + ", tel='" + tel
                + "', department=" + department + "}";
    }
}
```

Classe abstraite `DAO<T>` reprise de l'énoncé :

part1-jdbc/src/fr/isep/tp3/dao/DAO.java

```
package fr.isep.tp3.dao;

import java.sql.Connection;

public abstract class DAO<T> {
    protected Connection connect = null;

    public DAO(Connection connect) {
        this.connect = connect;
    }

    public abstract T find(int id);

    public abstract boolean create(T object);

    public abstract boolean update(T object);

    public abstract boolean delete(T object);
}
```

Exercice 10 – DeptDAO.find

`find` exécute un `SELECT ... WHERE deptno = ?` en `PreparedStatement` et construit un objet `Dept` ; il renvoie `null` si le département n'existe pas. J'ai également implémenté `create`, `update` et `delete` (testés dans l'exercice 12).

part1-jdbc/src/fr/isep/tp3/dao/DeptDAO.java

```
package fr.isep.tp3.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

/* Exercice 10 : DAO du département. */
public class DeptDAO extends DAO<Dept> {

    public DeptDAO(Connection connect) {
        super(connect);
    }

    @Override
    public Dept find(int id) {
```

```
        Dept dept = null;
        String sql = "SELECT deptno, dname, loc FROM dept WHERE deptno = ?";
        try (PreparedStatement ps = connect.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    dept = new Dept(rs.getInt("deptno"), rs.getString("dname"),
rs.getString("loc"));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return dept;
    }

    @Override
    public boolean create(Dept dept) {
        String sql = "INSERT INTO dept (deptno, dname, loc) VALUES (?, ?, ?)";
        try (PreparedStatement ps = connect.prepareStatement(sql)) {
            ps.setInt(1, dept.getDeptNo());
            ps.setString(2, dept.getDname());
            ps.setString(3, dept.getLoc());
            return ps.executeUpdate() == 1;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    @Override
    public boolean update(Dept dept) {
        String sql = "UPDATE dept SET dname = ?, loc = ? WHERE deptno = ?";
        try (PreparedStatement ps = connect.prepareStatement(sql)) {
            ps.setString(1, dept.getDname());
            ps.setString(2, dept.getLoc());
            ps.setInt(3, dept.getDeptNo());
            return ps.executeUpdate() == 1;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    @Override
    public boolean delete(Dept dept) {
        String sql = "DELETE FROM dept WHERE deptno = ?";
        try (PreparedStatement ps = connect.prepareStatement(sql)) {
            ps.setInt(1, dept.getDeptNo());
            return ps.executeUpdate() == 1;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }
}
```

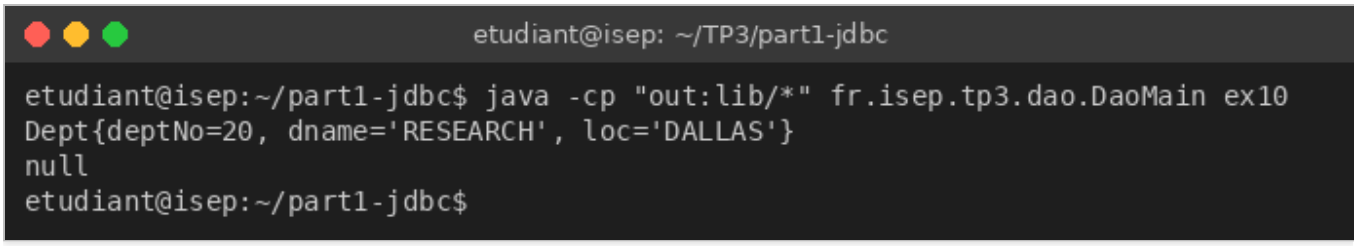


Figure 10 : find(20) renvoie le département RESEARCH ; find(99) renvoie null

Exercice 11 – EmpDAO.find (construction récursive)

En base, `mgr` est un entier ; en Java, c'est un `Emp`. Après avoir lu la ligne de l'employé, on rappelle donc `find(mgr)` **récursivement**, jusqu'à arriver au président dont le `mgr` est `NULL` (condition d'arrêt, testée avec `rs.wasNull()`). Le département est construit en déléguant au `DeptDAO`.

part1-jdbc/src/fr/isep/tp3/dao/EmpDAO.java

```java
package fr.isep.tp3.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Types;

/* Exercice 11 : DAO de l'employé, le manager est reconstruit récursivement. */
public class EmpDAO extends DAO<Emp> {

    private final DAO<Dept> deptDao;

    public EmpDAO(Connection connect) {
        super(connect);
        this.deptDao = new DeptDAO(connect);
    }

    @Override
    public Emp find(int id) {
        Emp emp = null;
        String sql = "SELECT * FROM emp WHERE empno = ?";
        try (PreparedStatement ps = connect.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    emp = new Emp();
                    emp.setEmpNo(rs.getLong("empno"));
                    emp.setEname(rs.getString("ename"));
                    emp.setEfirst(rs.getString("efirst"));
                    emp.setJob(rs.getString("job"));
                    emp.setHireDate(rs.getDate("hiredate"));
                    emp.setSal(rs.getInt("sal"));
                    emp.setComm(rs.getInt("comm"));
                    emp.setTel(rs.getString("tel"));

                    // le département est une clé étrangère -> on passe par le DAO Dept
                    emp.setDepartment(deptDao.find(rs.getInt("deptno")));

                    // le manager est un entier en base mais un Emp en Java :
                    // appel récursif de find() jusqu'au sommet de la hiérarchie (mgr NULL)
                    int mgrId = rs.getInt("mgr");
                    if (!rs.wasNull()) {
                        emp.setMgr(find(mgrId));
                    }
                }
```

```
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return emp;
    }

    @Override
    public boolean create(Emp emp) {
        String sql = "INSERT INTO emp (empno, ename, efirst, job, mgr, hiredate, sal, comm, tel,
deptno) "
                + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
        try (PreparedStatement ps = connect.prepareStatement(sql)) {
            ps.setLong(1, emp.getEmpNo());
            fill(ps, emp, 2);
            return ps.executeUpdate() == 1;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    @Override
    public boolean update(Emp emp) {
        String sql = "UPDATE emp SET ename = ?, efirst = ?, job = ?, mgr = ?, hiredate = ?, sal =
?, "
                + "comm = ?, tel = ?, deptno = ? WHERE empno = ?";
        try (PreparedStatement ps = connect.prepareStatement(sql)) {
            fill(ps, emp, 1);
            ps.setLong(10, emp.getEmpNo());
            return ps.executeUpdate() == 1;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    @Override
    public boolean delete(Emp emp) {
        String sql = "DELETE FROM emp WHERE empno = ?";
        try (PreparedStatement ps = connect.prepareStatement(sql)) {
            ps.setLong(1, emp.getEmpNo());
            return ps.executeUpdate() == 1;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    /* Remplit les 9 colonnes (hors empno) à partir de l'indice start. */
    private void fill(PreparedStatement ps, Emp emp, int start) throws SQLException {
        ps.setString(start, emp.getEname());
        ps.setString(start + 1, emp.getEfirst());
        ps.setString(start + 2, emp.getJob());
        if (emp.getMgr() != null) ps.setLong(start + 3, emp.getMgr().getEmpNo());
        else ps.setNull(start + 3, Types.INTEGER);
        if (emp.getHireDate() != null) ps.setDate(start + 4, new
java.sql.Date(emp.getHireDate().getTime()));
        else ps.setNull(start + 4, Types.DATE);
        ps.setInt(start + 5, emp.getSal());
        ps.setInt(start + 6, emp.getComm());
        ps.setString(start + 7, emp.getTel());
        if (emp.getDepartment() != null) ps.setInt(start + 8, emp.getDepartment().getDeptNo());
        else ps.setNull(start + 8, Types.INTEGER);
    }
}
```



Figure 11 : SMITH est reconstruit avec toute sa chaîne hiérarchique : SMITH → FORD → JONES → KING

Remarques / limites observées :

- Chaque niveau de hiérarchie déclenche 2 requêtes (employé + département) : c'est le problème classique du **« N+1 select »**. Pour une liste d'employés il vaudrait mieux une jointure ou un cache des objets déjà chargés (ce que fait JPA avec son contexte de persistance) ;
- si les données contenaient un cycle (A manager de B et B manager de A), la récursion serait infinie : un cache par identifiant règlerait aussi ce cas ;
- `getInt()` renvoie **0 pour un NULL** : sur la capture, `comm=0` pour SMITH alors que la valeur est NULL en base. Utiliser `Integer` dans le bean permettrait de distinguer les deux (c'est ce que fait le générateur de la partie V).

Exercice 12 – DAOFactory

La fabrique centralise l'instanciation des DAO : le code client ne fait plus jamais `new DeptDAO(...)` et ne manipule que le type abstrait `DAO<T>`. Changer d'implémentation (autre SGBD, pool de connexions...) ne demande de modifier que la fabrique.

part1-jdbc/src/fr/isep/tp3/dao/DAOFactory.java

```java
package fr.isep.tp3.dao;

import java.sql.Connection;

/* Exercice 12 : toutes les instanciations de DAO sont centralisées ici. */
public class DAOFactory {

    protected Connection connect;

    public DAOFactory(Connection connect) {
        this.connect = connect;
    }

    public DAO<Dept> getDeptDAO() {
        return new DeptDAO(connect);
    }

    public DAO<Emp> getEmpDAO() {
        return new EmpDAO(connect);
    }
}
```

part1-jdbc/src/fr/isep/tp3/dao/DaoMain.java

```java
package fr.isep.tp3.dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/* Tests des exercices 10, 11 et 12. Usage : java fr.isep.tp3.dao.DaoMain <ex10|ex11|ex12> */
public class DaoMain {

    public static void main(String[] args) throws SQLException {
```

```
        String exercise = args.length > 0 ? args[0] : "ex10";
        try (Connection connection = DriverManager.getConnection(
                "jdbc:postgresql://localhost/isep", "postgres", "postgres")) {

            if (exercise.equals("ex10")) {
                DAO<Dept> departmentDao = new DeptDAO(connection);
                Dept dept20 = departmentDao.find(20);
                System.out.println(dept20);
                System.out.println(departmentDao.find(99)); // département inexistant -> null
            } else if (exercise.equals("ex11")) {
                DAO<Emp> employeeDao = new EmpDAO(connection);
                Emp smith = employeeDao.find(7369);
                System.out.println(smith);
                // on remonte la chaîne hiérarchique construite récursivement
                Emp e = smith;
                StringBuilder chain = new StringBuilder();
                while (e != null) {
                    chain.append(e.getEname()).append(" (").append(e.getJob()).append(")");
                    e = e.getMgr();
                    if (e != null) chain.append(" -> ");
                }
                System.out.println("Hiérarchie : " + chain);
            } else if (exercise.equals("ex12")) {
                DAOFactory factory = new DAOFactory(connection);
                DAO<Dept> deptDao = factory.getDeptDAO();
                DAO<Emp> empDao = factory.getEmpDAO();

                System.out.println(deptDao.find(30));
                System.out.println(empDao.find(7499));

                // petit test CRUD complet sur un département
                Dept d = new Dept(50, "IT", "PARIS");
                System.out.println("create : " + deptDao.create(d) + " -> " + deptDao.find(50));
                d.setLoc("LYON");
                System.out.println("update : " + deptDao.update(d) + " -> " + deptDao.find(50));
                System.out.println("delete : " + deptDao.delete(d) + " -> " + deptDao.find(50));
            }
        }
    }
}
```



Figure 12 : utilisation de la factory + test CRUD complet (create / update / delete) sur un département 50

4. Partie III – Spring Boot & JPA

III.1 – Initialisation du projet Spring

Projet Maven `TP3_testjpa`, groupe `com.isep`, artefact `testjpa`, version `0.0.1-SNAPSHOT`, avec la classe `TestjpaApplication`, le `SimpleController` et le fichier `application.yml` (port 9090).

> **Adaptation de version :** l'énoncé utilise Spring Boot `2.2.5.RELEASE` et Java 8/16. Or la 2.2.x n'est supportée qu'avec Java 8 à 13, et ma machine a un JDK 21 (ses versions de Spring 5.2 / Hibernate 5.4 ne sont pas prévues pour lire des classes Java 21). J'ai donc pris **Spring Boot 2.7.18**, la dernière version 2.x : elle garde `javax.persistence`, donc tout le code de l'énoncé reste valable sans modification (la version 3.x aurait imposé `jakarta.persistence`).

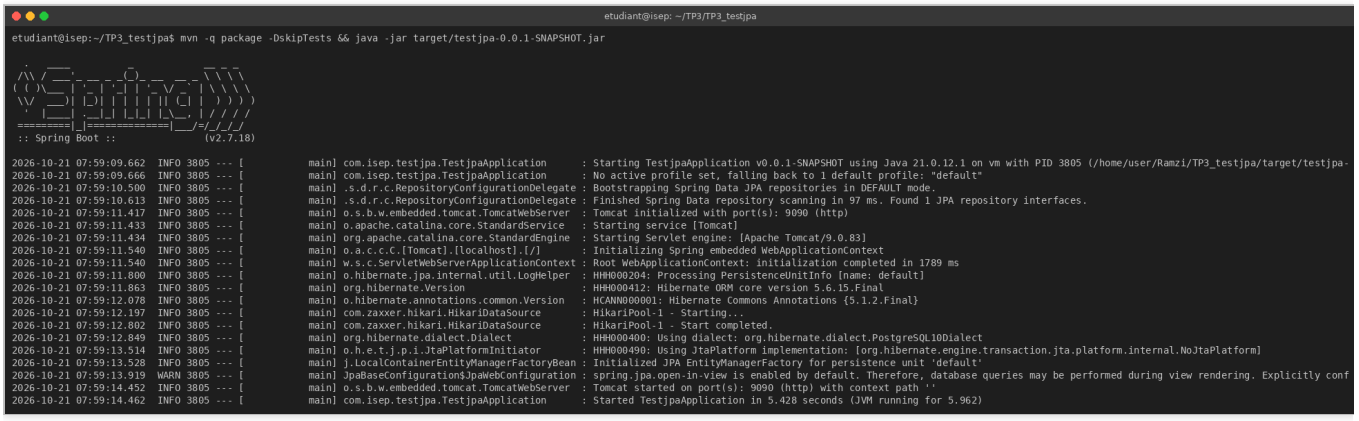


Figure 13 : démarrage de l'application : Tomcat sur le port 9090, connexion Hikari à PostgreSQL

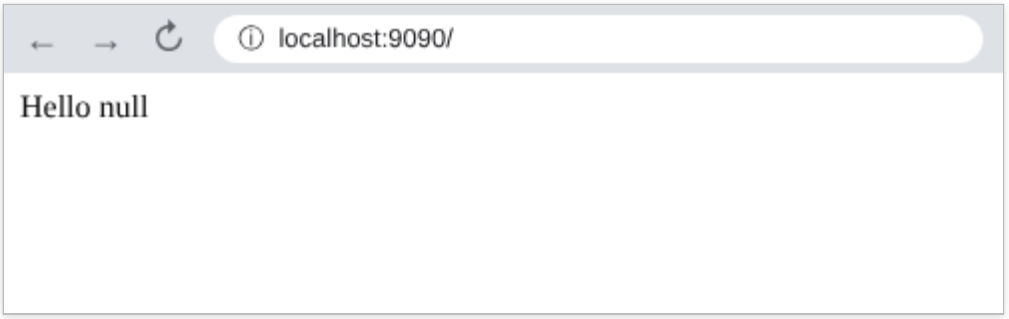


Figure 14 : localhost:9090 sans paramètre



Figure 15 : localhost:9090/?name=Bob%20Marley

Remarque : la deuxième version du contrôleur de l'énoncé utilise `@Param("name")` de `org.springframework.data.repository.query`. Cette annotation sert à nommer les paramètres des requêtes des *repositories* Spring Data, pas à lire un paramètre HTTP ; cela fonctionne uniquement parce que le nom de la variable Java est aussi `name`. J'ai gardé `@RequestParam(value = "name", required = false)`, qui est l'annotation correcte.

III.2 – Activation de JPA et connexion à la base

Ajout de `spring-boot-starter-data-jpa` et du driver PostgreSQL dans le `pom.xml` (attention, l'énoncé contient une coquille : `org.postresql` au lieu de `org.postgresql`, ce qui empêche Maven de trouver la dépendance), puis de la configuration de la `DataSource` dans `application.yml`. Le démarrage sans erreur (figure précédente, lignes `HikariPool-1 - Start completed` et `Using dialect: PostgreSQL10Dialect`) confirme que la connexion fonctionne. On remarque au passage que Spring Boot utilise **HikariCP** comme pool de connexions par défaut (lien avec la partie IV).

III.3 – Lecture des données

Avec l'entité `Emp` de l'énoncé (getter/setter uniquement pour `empno`), l'endpoint `/employees` fonctionne, mais Jackson ne sérialise que les propriétés qui ont un getter : on n'obtient que les identifiants.

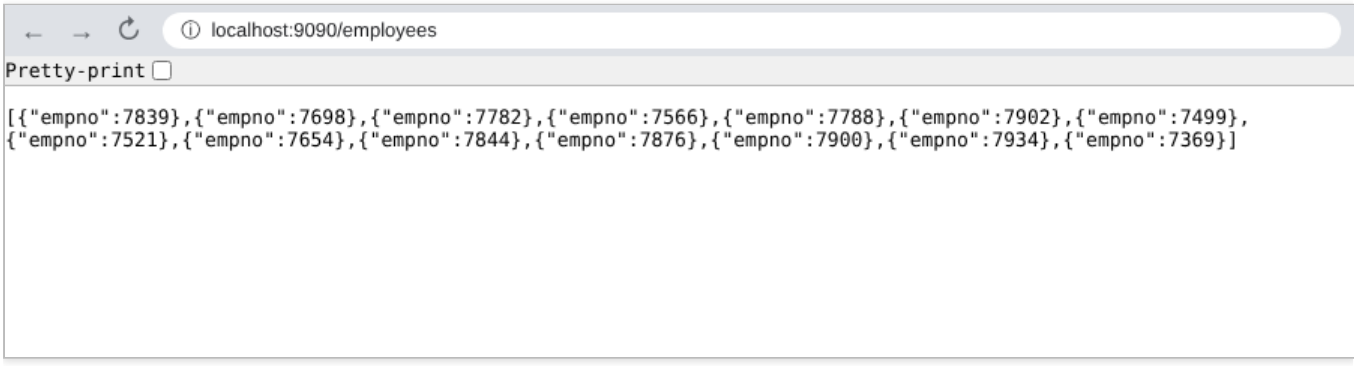


Figure 16 : /employees avec l'entité de l'énoncé : seul empno apparaît

J'ai ensuite ajouté **Lombok** (`@Data` génère getters, setters, `equals/hashCode` et `toString` à la compilation) et mappé toutes les colonnes de la table. J'ai utilisé Lombok 1.18.34 car la 1.18.12 proposée dans l'énoncé ne fonctionne pas avec un JDK 21.

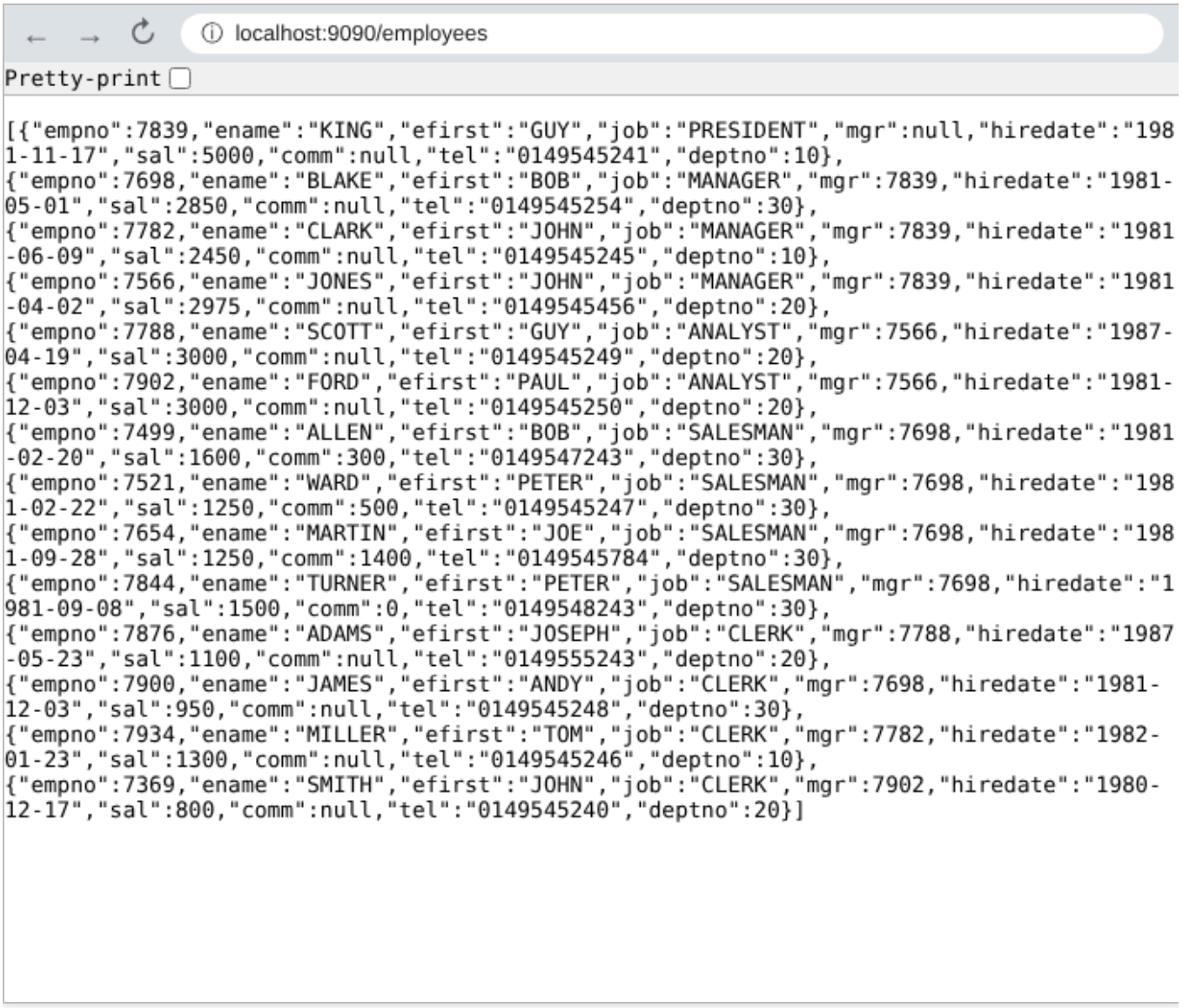


Figure 17 : /employees avec @Data : tous les attributs sont renvoyés en JSON

III.4 – Création d'un objet via JPA + Swagger

Ajout de la méthode `@PostMapping("/employees")` qui appelle `empRepository.save(emp)`. Au premier test, la création échoue :

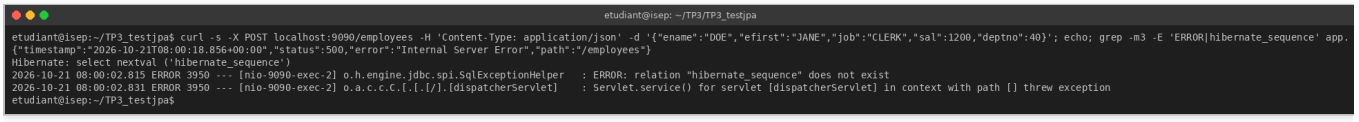


Figure 18 : erreur 500 : la séquence hibernate_sequence n'existe pas

> **Explication :** l'entité de l'énoncé porte `@GeneratedValue` sans stratégie, donc `GenerationType.AUTO`. Avec PostgreSQL, Hibernate 5 choisit alors une séquence nommée `hibernate_sequence` (`select nextval('hibernate_sequence')`)... qui n'existe pas dans notre schéma, puisque `empno` est une clé saisie manuellement (pas de `SERIAL` / `IDENTITY`). **Correction :** j'ai retiré `@GeneratedValue` ; l'`empno` est fourni par le client dans le JSON, comme dans la base d'origine. (Autre solution possible : créer une séquence et utiliser `@GeneratedValue(strategy = GenerationType.SEQUENCE)` avec `@SequenceGenerator`.)

Pour Swagger, Springfox 2.9.2 n'est pas compatible avec le nouveau moteur de correspondance d'URL de Spring Boot 2.6+ ; il faut ajouter `spring.mvc.pathmatch.matching-strategy: ant_path_matcher` dans `application.yml`, sinon l'application ne démarre pas.

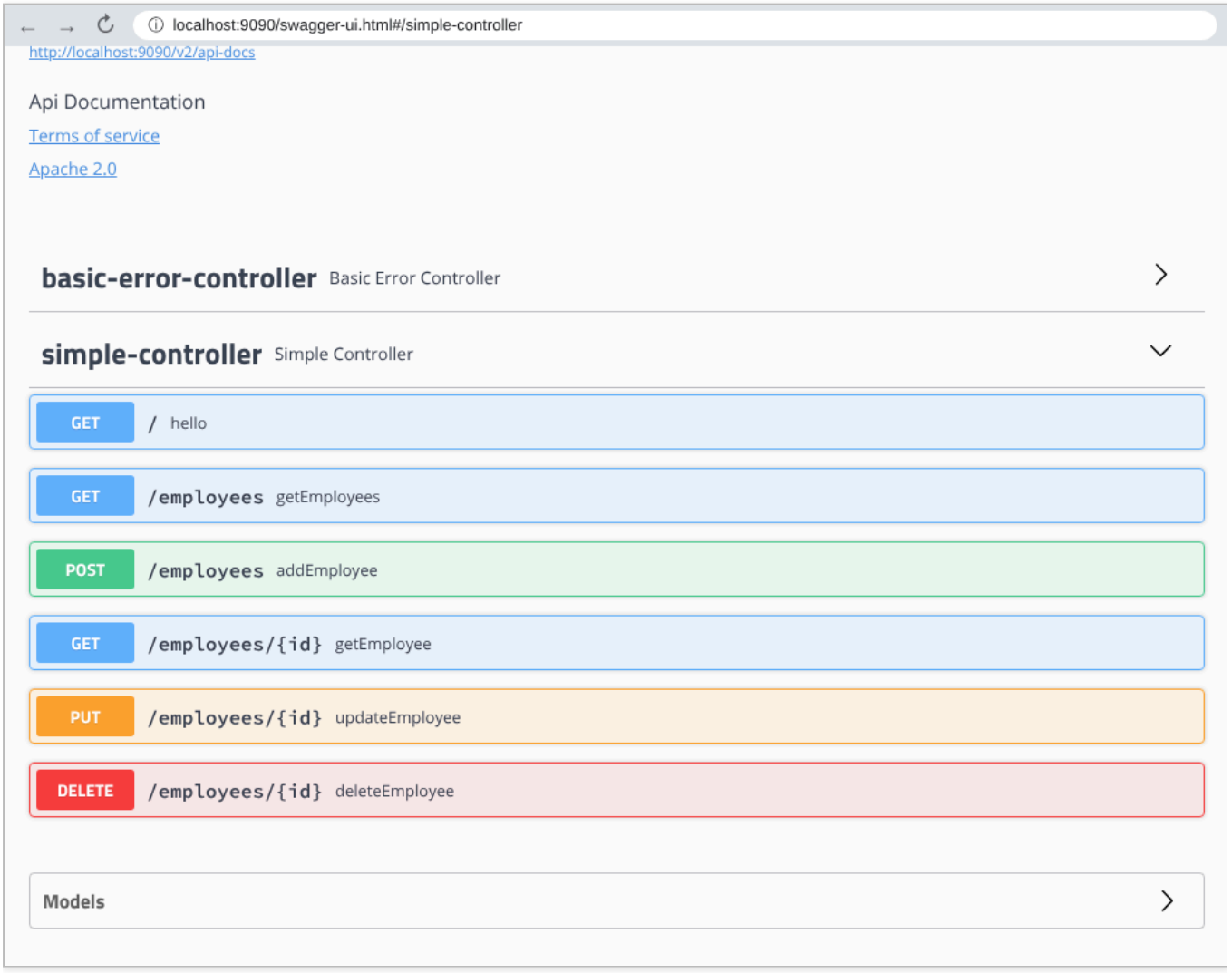


Figure 19 : Swagger UI : tous les endpoints du contrôleur sont détectés automatiquement

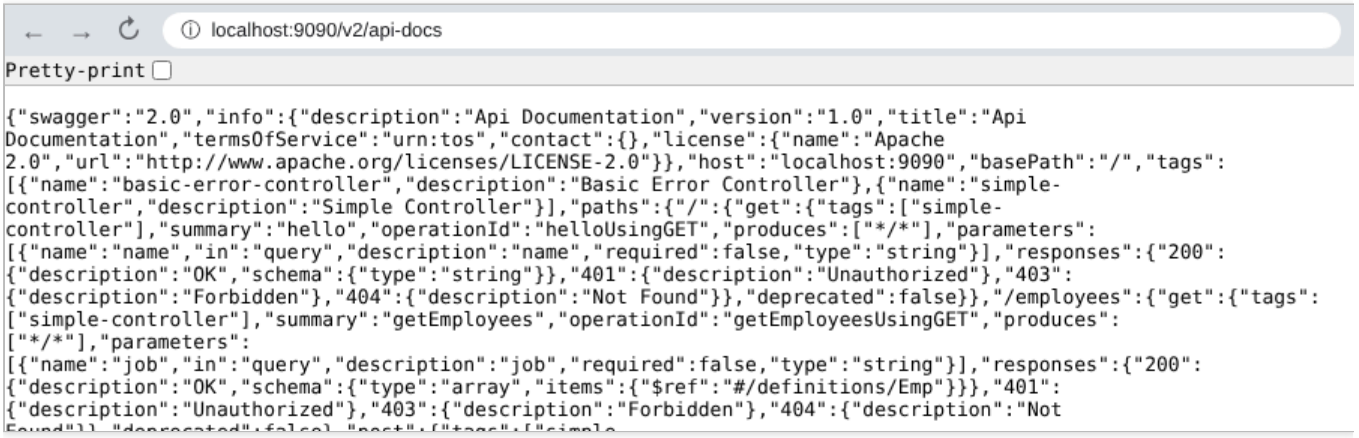


Figure 20 : le JSON /v2/api-docs généré, sur lequel s'appuie l'interface

III.5 – GET by ID / GET all / Update / Delete via JPA et REST

Conventions REST respectées :

Méthode	URL	Action	Codes de retour
GET	`/employees`	liste (filtre optionnel `?job=`)	200
GET	`/employees/{id}`	un employé	200, 404 si inconnu
POST	`/employees`	création	201 + en-tête `Location`, 400 sans empno, 409 si déjà existant
PUT	`/employees/{id}`	mise à jour complète	200, 404 si inconnu
DELETE	`/employees/{id}`	suppression	204, 404 si inconnu

Ressource au **pluriel**, identifiant dans le chemin, **verbe HTTP** pour l'action (pas de `/deleteEmployee`), et des codes HTTP significatifs grâce à `ResponseEntity`. Le filtre par métier utilise une **requête dérivée** Spring Data (`findByJob`) : la requête SQL est générée à partir du nom de la méthode, sans écrire de SQL.



Figure 21 : script de test curl : tous les cas (succès et erreurs) renvoient le bon code HTTP

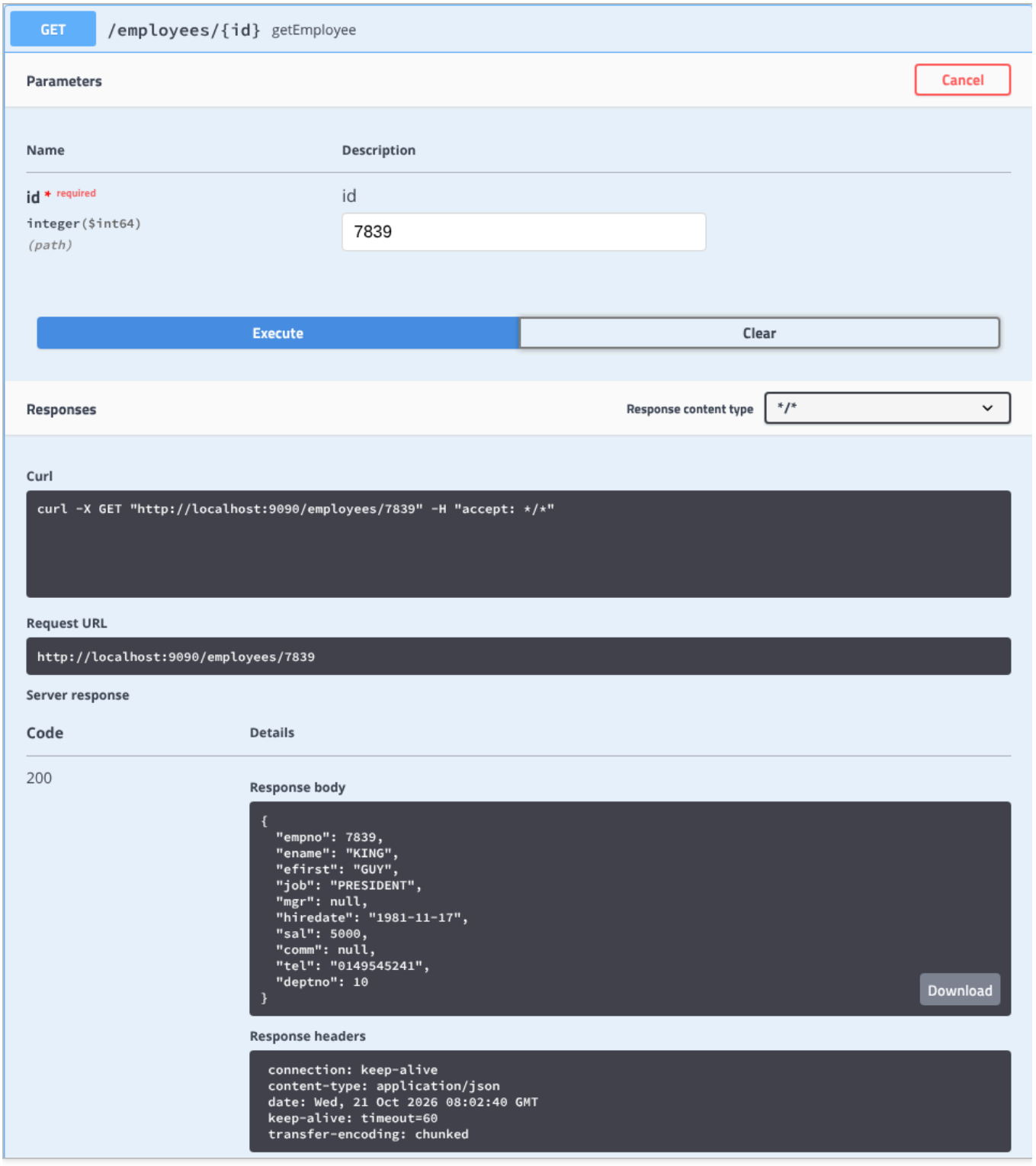


Figure 22 : test de GET /employees/{id} directement depuis Swagger UI

Code complet du projet (copié comme demandé)

TP3_testjpa/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-parent</artifactId>
        <!-- 2.2.5.RELEASE dans l'énoncé : ne démarre pas avec un JDK 21, on prend la dernière 2.x
-->
        <version>2.7.18</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>

    <groupId>com.isep</groupId>
    <artifactId>testjpa</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>testjpa</name>
    <description>Demo project for Spring Boot</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <version>42.7.3</version>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <version>1.18.34</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>io.springfox</groupId>
            <artifactId>springfox-swagger2</artifactId>
            <version>2.9.2</version>
        </dependency>
        <dependency>
            <groupId>io.springfox</groupId>
            <artifactId>springfox-swagger-ui</artifactId>
            <version>2.9.2</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

TP3_testjpa/src/main/resources/application.yml

```
server.port: 9090

spring.datasource.url: jdbc:postgresql://localhost/isep
spring.datasource.username: postgres
spring.datasource.password: postgres
spring.datasource.driver-class-name: org.postgresql.Driver

# affiche le SQL généré par Hibernate dans la console
spring.jpa.show-sql: true
# on ne laisse pas Hibernate modifier le schéma existant
spring.jpa.hibernate.ddl-auto: none

# Springfox 2.9.2 n'est pas compatible avec le PathPatternParser par défaut de Spring Boot 2.6+
spring.mvc.pathmatch.matching-strategy: ant_path_matcher
```

TP3_testjpa/src/main/java/com/isep/testjpa/TestjpaApplication.java

```
package com.isep.testjpa;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TestjpaApplication {

    public static void main(String[] args) {
        SpringApplication.run(TestjpaApplication.class, args);
    }

}
```

TP3_testjpa/src/main/java/com/isep/testjpa/model/Emp.java

```
package com.isep.testjpa.model;

import lombok.Data;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import java.time.LocalDate;

@Data
@Entity
public class Emp {
    // pas de @GeneratedValue : empno n'est pas auto-incrémenté dans la table,
    // avec la stratégie AUTO Hibernate cherche une séquence "hibernate_sequence" qui n'existe pas
    @Id
    private Long empno;
    @Column(name = "ename")
    private String ename;
    @Column(name = "efirst")
    private String efirst;
    @Column(name = "job")
    private String job;
    @Column(name = "mgr")
    private Long mgr;
    @Column(name = "hiredate")
    private LocalDate hiredate;
    @Column(name = "sal")
    private Long sal;
    @Column(name = "comm")
    private Long comm;
    @Column(name = "tel")
    private String tel;
```

```
    @Column(name = "deptno")
    private Long deptno;
}
```

TP3_testjpa/src/main/java/com/isep/testjpa/repository/EmpRepository.java

```
package com.isep.testjpa.repository;

import com.isep.testjpa.model.Emp;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface EmpRepository extends JpaRepository<Emp, Long> {

    // requête dérivée du nom de la méthode : SELECT * FROM emp WHERE job = ?
    List<Emp> findByJob(String job);

}
```

TP3_testjpa/src/main/java/com/isep/testjpa/controller/SimpleController.java

```
package com.isep.testjpa.controller;

import com.isep.testjpa.repository.EmpRepository;
import com.isep.testjpa.model.Emp;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.List;

@RestController
public class SimpleController {

    @Autowired
    private EmpRepository empRepository;

    @RequestMapping(value = "/", method = RequestMethod.GET)
    public String hello(@RequestParam(value = "name", required = false) String name) {
        return "Hello " + name;
    }

    /* GET /employees (option : ?job=CLERK pour filtrer) */
    @GetMapping("/employees")
    public List<Emp> getEmployees(@RequestParam(value = "job", required = false) String job) {
        if (job != null) {
            return empRepository.findByJob(job.toUpperCase());
        }
        return empRepository.findAll();
    }
```

```
    /* GET /employees/{id} : 200 + employé, ou 404 */
    @GetMapping("/employees/{id}")
    public ResponseEntity<Emp> getEmployee(@PathVariable Long id) {
        return empRepository.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    /* POST /employees : 201 + header Location, 400 si pas d'empno, 409 si déjà existant */
    @PostMapping("/employees")
    public ResponseEntity<Emp> addEmployee(@RequestBody Emp emp) {
        if (emp.getEmpno() == null) {
            return ResponseEntity.badRequest().build();
        }
        if (empRepository.existsById(emp.getEmpno())) {
            return ResponseEntity.status(HttpStatus.CONFLICT).build();
        }
        Emp saved = empRepository.save(emp);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(saved.getEmpno()).toUri();
        return ResponseEntity.created(location).body(saved);
    }

    /* PUT /employees/{id} : remplace l'employé, 200 ou 404 */
    @PutMapping("/employees/{id}")
    public ResponseEntity<Emp> updateEmployee(@PathVariable Long id, @RequestBody Emp emp) {
        if (!empRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        emp.setEmpno(id); // l'id de l'URL fait foi
        return ResponseEntity.ok(empRepository.save(emp));
    }

    /* DELETE /employees/{id} : 204 ou 404 */
    @DeleteMapping("/employees/{id}")
    public ResponseEntity<Void> deleteEmployee(@PathVariable Long id) {
        if (!empRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        empRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }

}
```

TP3_testjpa/src/main/java/com/isep/testjpa/configuration/SpringFoxConfig.java

```
package com.isep.testjpa.configuration;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import springfox.documentation.builders.PathSelectors;
import springfox.documentation.builders.RequestHandlerSelectors;
import springfox.documentation.spi.DocumentationType;
import springfox.documentation.spring.web.plugins.Docket;
import springfox.documentation.swagger2.annotations.EnableSwagger2;

@Configuration
@EnableSwagger2
public class SpringFoxConfig {
    @Bean
    public Docket api() {
        return new Docket(DocumentationType.SWAGGER_2)
                .select()
                .apis(RequestHandlerSelectors.any())
                .paths(PathSelectors.any())
```

```
                .build();
    }
}
```

TP3_testjpa/test_api.sh

```
#!/bin/bash
# Tests de l'API REST /employees (exercices III.4 et III.5)
API=http://localhost:9090/employees
H='Content-Type: application/json'
c() { curl -s -w "   -> HTTP %{http_code}\n" "$@"; }

echo "### GET $API/7788  (get by id)"
c $API/7788
echo "### GET $API/1234  (id inexistant)"
c $API/1234
echo "### POST $API  (création)"
c -X POST $API -H "$H" -d
'{"empno":8000,"ename":"DOE","efirst":"JANE","job":"CLERK","mgr":7782,"hiredate":"2026-10-
21","sal":1200,"deptno":10}'
echo "### POST $API  (même empno -> conflit)"
c -X POST $API -H "$H" -d '{"empno":8000,"ename":"DOE"}'
echo "### PUT $API/8000  (mise à jour : augmentation + changement de département)"
c -X PUT $API/8000 -H "$H" -d
'{"ename":"DOE","efirst":"JANE","job":"ANALYST","mgr":7566,"hiredate":"2026-10-
21","sal":3100,"deptno":20}'
echo "### GET $API?job=analyst  (filtre, on n'affiche que empno/ename)"
curl -s "$API?job=analyst" | grep -o '"empno":[0-9]*,"ename":"[A-Z]*"'
echo "### DELETE $API/8000"
c -X DELETE $API/8000
echo "### GET $API/8000  (après suppression)"
c $API/8000
```

Comparaison avec la partie II : le `EmpRepository` (une interface vide de 3 lignes) remplace tout le `EmpDAO` écrit à la main : Spring Data génère l'implémentation (`findAll`, `findById`, `save`, `deleteById`...), Hibernate fait le mapping ligne ↔ objet grâce aux annotations, et la gestion des connexions et des transactions est automatique. C'est exactement le rôle du pattern DAO, mais fourni par le framework.

5. Partie IV – Pool de connexions (bonus)

IV.1 – Pool de connexions « fait main »

Au démarrage, le pool ouvre N connexions via une `DataSource` (`PGSimpleDataSource`) au lieu du `DriverManager` et les range dans une `ArrayBlockingQueue`. Un client prend une connexion avec `getConnection()` et la rend avec `release()` au lieu de la fermer. La `BlockingQueue` est thread-safe : si toutes les connexions sont prises, le thread appelant attend qu'une connexion soit rendue, ce qui borne le nombre de connexions ouvertes sur le serveur.

part1-jdbc/src/fr/isep/tp3/pool/ConnectionPool.java

```
package fr.isep.tp3.pool;

import org.postgresql.ds.PGSimpleDataSource;

import javax.sql.DataSource;
import java.sql.Connection;
import java.sql.SQLException;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

/*
 * Exercice IV.1 (bonus) : pool de connexions "fait main".
 * Au démarrage on ouvre N connexions via une DataSource et on les range dans une file.
 * Un client prend une connexion (getConnection), fait sa requête et la rend (release).
 * La BlockingQueue gère le multi-thread : si le pool est vide, le client attend.
 */
public class ConnectionPool {

    private final BlockingQueue<Connection> pool;

    public ConnectionPool(String url, String user, String password, int size) throws SQLException {
        PGSimpleDataSource ds = new PGSimpleDataSource();
        ds.setUrl(url);
        ds.setUser(user);
        ds.setPassword(password);
        this.pool = new ArrayBlockingQueue<>(size);
        for (int i = 0; i < size; i++) {
            pool.add(createConnection(ds));
        }
    }

    private static Connection createConnection(DataSource ds) throws SQLException {
        return ds.getConnection();
    }

    public Connection getConnection() throws InterruptedException {
        return pool.take();
    }

    public void release(Connection connection) {
        if (connection != null) {
            pool.offer(connection);
        }
    }

    public int available() {
        return pool.size();
    }
```

```
    public void shutdown() throws SQLException {
        for (Connection c : pool) {
            c.close();
        }
        pool.clear();
    }
}
```

IV.2 – BoneCP

Grâce à l'interface `ConnectionProvider`, passer du pool maison à BoneCP ne change que quelques lignes : c'est l'intérêt d'avoir isolé l'obtention des connexions. Avec BoneCP, `close()` ne ferme pas la connexion mais la rend au pool.

part1-jdbc/src/fr/isep/tp3/pool/PoolMain.java

```
package fr.isep.tp3.pool;

import com.jolbox.bonecp.BoneCP;
import com.jolbox.bonecp.BoneCPConfig;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

/*
 * Bonus partie IV : comparaison sans pool / pool maison / BoneCP.
 * On simule NB_REQUESTS "clients" servis par 10 threads, chacun fait un SELECT.
 */
public class PoolMain {

    static final String URL = "jdbc:postgresql://localhost/isep";
    static final String USER = "postgres";
    static final String PASS = "postgres";
    static final int NB_REQUESTS = 500;
    static final int NB_THREADS = 10;

    interface ConnectionProvider {
        Connection get() throws Exception;

        void giveBack(Connection c) throws Exception;
    }

    public static void main(String[] args) throws Exception {
        // 1) Sans pool : une nouvelle connexion par requête (DriverManager)
        long t = run(new ConnectionProvider() {
            public Connection get() throws SQLException {
                return DriverManager.getConnection(URL, USER, PASS);
            }

            public void giveBack(Connection c) throws SQLException {
                c.close();
            }
        });
        System.out.println("Sans pool (DriverManager)      : " + NB_REQUESTS + " requêtes en " + t
+ " ms");

        // 2) Exercice IV.1 : pool maison
```

```
        ConnectionPool pool = new ConnectionPool(URL, USER, PASS, NB_THREADS);
        System.out.println("Pool maison créé, connexions disponibles : " + pool.available());
        t = run(new ConnectionProvider() {
            public Connection get() throws InterruptedException {
                return pool.getConnection();
            }

            public void giveBack(Connection c) {
                pool.release(c);
            }
        });
        System.out.println("Pool maison (IV.1)             : " + NB_REQUESTS + " requêtes en " + t
+ " ms");
        System.out.println("Connexions disponibles après le test : " + pool.available());
        pool.shutdown();

        // 3) Exercice IV.2 : BoneCP
        BoneCPConfig config = new BoneCPConfig();
        config.setJdbcUrl(URL);
        config.setUsername(USER);
        config.setPassword(PASS);
        config.setMinConnectionsPerPartition(5);
        config.setMaxConnectionsPerPartition(10);
        config.setPartitionCount(1);
        BoneCP boneCP = new BoneCP(config);
        t = run(new ConnectionProvider() {
            public Connection get() throws SQLException {
                return boneCP.getConnection();
            }

            public void giveBack(Connection c) throws SQLException {
                c.close(); // avec BoneCP, close() rend la connexion au pool
            }
        });
        System.out.println("BoneCP (IV.2)                  : " + NB_REQUESTS + " requêtes en " + t
+ " ms");
        System.out.println("BoneCP : connexions créées = " + boneCP.getTotalCreatedConnections());
        boneCP.shutdown();
    }

    static long run(ConnectionProvider provider) throws InterruptedException {
        ExecutorService executor = Executors.newFixedThreadPool(NB_THREADS);
        long start = System.currentTimeMillis();
        for (int i = 0; i < NB_REQUESTS; i++) {
            final int empno = (i % 2 == 0) ? 7369 : 7839;
            executor.submit(() -> {
                Connection c = null;
                try {
                    c = provider.get();
                    try (PreparedStatement ps = c.prepareStatement("SELECT ename FROM emp WHERE
empno = ?")) {
                        ps.setInt(1, empno);
                        try (ResultSet rs = ps.executeQuery()) {
                            rs.next();
                        }
                    }
                } catch (Exception e) {
                    e.printStackTrace();
                } finally {
                    try {
                        provider.giveBack(c);
                    } catch (Exception ignore) {
                    }
                }
            });
        }
        executor.shutdown();
```

```
        executor.awaitTermination(1, TimeUnit.MINUTES);
        return System.currentTimeMillis() - start;
    }
}
```

Mesures

Test : 500 requêtes `SELECT` exécutées par 10 threads en parallèle.



Figure 23 : comparaison sans pool / pool maison / BoneCP

> Sans pool, les 500 requêtes prennent environ **5,2 s** ; avec un pool, environ **50 à 75 ms**, soit un facteur d'environ **100**. Presque tout le temps est passé à **ouvrir les connexions** (connexion TCP, création d'un processus côté PostgreSQL, authentification SCRAM-SHA-256 volontairement coûteuse), et non à exécuter la requête elle-même. Le pool paie ce coût une seule fois au démarrage. On voit aussi que BoneCP n'a créé que 7 connexions pour servir les 10 threads.
>
> BoneCP n'est plus maintenu : son auteur conseille lui-même **HikariCP**, qui est justement le pool par défaut de Spring Boot (cf. logs de la partie III) et qui se règle dans `application.yml` (`spring.datasource.hikari.maximum-pool-size`, etc.).

6. Partie V – Générateur de DAO (bonus)

Le générateur lit le **dictionnaire de données** de PostgreSQL (vue standard `information_schema.columns`, plus `table_constraints` / `key_column_usage` pour repérer la clé primaire), puis écrit pour chaque table une classe POJO et une classe DAO (méthode `find` implémentée, les autres laissées en `TODO`).

- **V.1** : liste de toutes les colonnes avec leur type et la clé primaire ;
- **V.2 / V.3** : création des fichiers `.java` des POJO (attributs, getters/setters, `toString`) avec conversion des types SQL → Java ;
- **V.4** : création des DAO qui héritent de la classe `DAO<T>` de la partie II.

```
etudiant@isep: ~/TP3/part1-jdbc
etudiant@isep:~/part1-jdbc$ java -cp "out:lib/*" fr.isep.tp3.generator.DaoGenerator generated
TABLE dept
   deptno     integer              PK
   dname      character varying
   loc        character varying
TABLE emp
   empno      integer              PK
   ename      character varying
   efirst     character varying
   job        character varying
   mgr        integer
   hiredate   date
   sal        integer
   comm       integer
   tel        character varying
   deptno     integer
Generated generated/fr/isep/tp3/generated/Dept.java and generated/fr/isep/tp3/generated/DeptDAO.java
Generated generated/fr/isep/tp3/generated/Emp.java and generated/fr/isep/tp3/generated/EmpDAO.java
etudiant@isep:~/part1-jdbc$
```

Figure 24 : V.1 : lecture du dictionnaire de données puis génération des 4 fichiers

```
etudiant@isep: ~/TP3/part1-jdbc
etudiant@isep:~/part1-jdbc$ javac -encoding UTF-8 -cp "out:lib/*" -d out $(find generated test-generated -name "*.java") && java -cp "out:lib/*" fr.is
Dept{deptno=10, dname=ACCOUNTING, loc=NEW YORK}
Emp{empno=7839, ename=KING, efirst=GUY, job=PRESIDENT, mgr=null, hiredate=1981-11-17, sal=5000, comm=null, tel=0149545241, deptno=10}
etudiant@isep:~/part1-jdbc$
```

Figure 25 : les classes générées compilent et fonctionnent

Extrait du générateur (lecture du dictionnaire) et exemple de classe générée :

part1-jdbc/src/fr/isep/tp3/generator/DaoGenerator.java (extrait)

```
static Map<String, List<Column>> readDictionary(Connection c) throws SQLException {
    String sql = "SELECT c.table_name, c.column_name, c.data_type, "
            + "  (SELECT count(*) FROM information_schema.key_column_usage k "
            + "     JOIN information_schema.table_constraints tc ON tc.constraint_name =
k.constraint_name "
            + "    WHERE tc.constraint_type = 'PRIMARY KEY' AND k.table_name = c.table_name "
            + "      AND k.column_name = c.column_name) AS is_pk "
            + "FROM information_schema.columns c "
            + "WHERE c.table_schema = ? ORDER BY c.table_name, c.ordinal_position";
    Map<String, List<Column>> tables = new LinkedHashMap<>();
    try (PreparedStatement ps = c.prepareStatement(sql)) {
        ps.setString(1, "public");
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                tables.computeIfAbsent(rs.getString("table_name"), k -> new ArrayList<>())
```

```
                        .add(new Column(rs.getString("column_name"), rs.getString("data_type"),
rs.getInt("is_pk") > 0));
            }
        }
    }
    return tables;
}
```

part1-jdbc/generated/fr/isep/tp3/generated/DeptDAO.java

```java
package fr.isep.tp3.generated;

import fr.isep.tp3.dao.DAO;

import java.sql.*;

/* Generated by DaoGenerator - do not edit */
public class DeptDAO extends DAO<Dept> {

    public DeptDAO(Connection connect) {
        super(connect);
    }

    @Override
    public Dept find(int id) {
        String sql = "SELECT * FROM dept WHERE deptno = ?";
        try (PreparedStatement ps = connect.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) return null;
                Dept o = new Dept();
                o.setDeptno((Integer) rs.getObject("deptno"));
                o.setDname((String) rs.getObject("dname"));
                o.setLoc((String) rs.getObject("loc"));
                return o;
            }
        } catch (SQLException e) {
            e.printStackTrace();
            return null;
        }
    }

    @Override
    public boolean create(Dept object) {
        throw new UnsupportedOperationException("TODO");
    }

    @Override
    public boolean update(Dept object) {
        throw new UnsupportedOperationException("TODO");
    }

    @Override
    public boolean delete(Dept object) {
        throw new UnsupportedOperationException("TODO");
    }

}
```

7. Conclusion

Ce TP montre l'évolution de l'accès aux données en Java :

1. **JDBC brut** : contrôle total mais beaucoup de code, et des risques réels (injection SQL, fuites de connexions). Le `PreparedStatement` est indispensable dès qu'une valeur vient de l'utilisateur, et un identifiant (nom de table) doit être validé par liste blanche ;
2. **DAO + Factory** : la persistance est isolée du métier, qui manipule des objets ; mais le mapping (notamment les relations comme le manager) reste à écrire et à optimiser à la main ;
3. **Spring Boot + JPA** : le framework fournit le DAO (repository), le mapping, le pool de connexions (HikariCP) et l'exposition REST, avec très peu de code. Il faut néanmoins comprendre ce qu'il fait pour corriger les problèmes rencontrés (séquence `hibernate_sequence`, compatibilité des versions, getters manquants) ;
4. le **pool de connexions** divise le temps de réponse par environ 100 dans notre test, ce qui justifie son usage systématique côté serveur.